Zavyalov. Work 4

Практическое задание 8. Анализ цепочки ошибок для конкретной системы

Выбранный вариант: А — интернет-магазин, обслуживающий 10 000 пользователей в день.

Шаг 1. Описание инцидента

Во время вечернего пика нагрузки часть покупателей оформляет заказы, но после нажатия кнопки «Оформить заказ» сайт долго отвечает, а некоторые заказы не появляются в личном кабинете. При этом отдельные покупатели видят сообщение об ошибке и не понимают, был ли заказ создан.

Симптом: задержка оформления заказа и отсутствие подтверждения или отображения части заказов.
Форма проявления: явная — сообщение об ошибке и длительное ожидание; для части пользователей скрытая — заказ не отображается сразу, и проблема обнаруживается позднее.
Последствия: повторные попытки оформления, риск дублирования заказов, потеря части продаж, рост обращений в поддержку и снижение доверия клиентов.

Шаг 2. Построение цепочки ошибок

1. Дестабилизирующий фактор — резкий рост числа одновременных запросов во время вечернего пика; нагрузка превышает возможности обработки запросов к базе данных.
2. Первичная ошибка — в системе не предусмотрено достаточное управление нагрузкой и ограничение времени ожидания при обращении к базе данных.
3. Вторичная ошибка — запросы оформления заказа задерживаются или завершаются тайм-аутом, а приложение не всегда корректно обрабатывает этот результат.
4. Третичная ошибка — пользователь не получает достоверного статуса заказа и повторно отправляет запрос; в отдельных случаях возникают дублирующиеся попытки или заказ не отображается в интерфейсе.
5. Последствие — потеря продаж, дополнительная нагрузка на поддержку, ухудшение пользовательского опыта и репутационные потери.

Шаг 3. Метод «5 почему»

1. Почему часть покупателей не видит подтверждение заказа? — Запрос оформления завершается тайм-аутом или статус заказа не возвращается вовремя.

2. Почему возникает тайм-аут? — При высокой нагрузке запросы к базе данных обрабатываются слишком долго.
3. Почему запросы обрабатываются слишком долго? — Одновременные обращения превышают расчётную пропускную способность базы данных и приложения.
4. Почему система не справляется с пиковыми обращениями? — Нет достаточного ограничения нагрузки, контроля очередей и проверенного сценария работы при перегрузке.
5. Почему эти меры не были предусмотрены? — Пиковая нагрузка и поведение системы при превышении лимитов не были в достаточной степени проверены при проектировании и нагрузочном тестировании.
Корневая причина: недостаточная проработка и проверка обработки пиковой нагрузки, включая управление очередями и тайм-аутами.

Шаг 4. Меры предотвращения

1. Ограничить число одновременно обрабатываемых запросов и настроить очереди, тайм-ауты и контролируемые повторные попытки. Тип меры: программная. Вид избыточности: временная — сохранение состояния и возможность безопасного повторного выполнения операции; также функциональная — распределение обработки между компонентами. Эффект: ограничивает перегрузку и не позволяет задержкам бесконтрольно распространяться по системе.

2. Реализовать идемпотентность оформления заказа: каждому запросу присваивать уникальный ключ и не создавать второй заказ при повторной отправке того же запроса. Тип меры: программная. Вид избыточности: информационная — дополнительный идентификатор и проверка состояния операции. Эффект: разрывает цепочку «тайм-аут → повторный запрос → дублирование заказа».

3. Масштабировать приложение и критические компоненты базы данных, предусмотреть резервирование там, где оно применимо. Тип меры: техническая. Вид избыточности: структурная. Эффект: распределяет нагрузку и снижает вероятность того, что отказ одного компонента остановит оформление заказов.

4. Проводить нагрузочные и отказные тесты для обычной и пиковой нагрузки, а также регулярно проверять сценарии тайм-аутов и повторных запросов. Тип меры: организационная. Вид избыточности: программная — независимые тестовые сценарии и проверки. Эффект: позволяет обнаружить слабые места до эксплуатации и подтвердить, что цепочка ошибок прерывается.

Вывод

Инцидент начинается с недостаточной готовности системы к пиковой нагрузке. Первичная ошибка вызывает задержки и тайм-ауты, затем приводит к отсутствию достоверного статуса заказа и повторным запросам. Управление нагрузкой, идемпотентность, резервирование и регулярное тестирование помогают разорвать цепочку и уменьшить последствия.